

CLASE 5

# Datos, JSON y paquetes externos

La View dejará de contener los datos y aprenderemos a justificar cada dependencia

JSON →  
Python →  
Contexto →  
Template

## El catálogo funciona, pero los datos están mezclados

- La URL conduce correctamente a catalogo.
- La View prepara una lista de diccionarios.
- El Template representa cada dispositivo.
- Modificar datos obliga a editar código Python.

**Separaremos la fuente de datos sin cambiar el contrato con el Template.**

¿Qué elemento debería seguir recibiendo el Template aunque cambie la fuente?

**El contexto debe conservar una colección con la estructura que el Template espera.**

# Hoy separaremos, cargaremos y comprobaremos datos

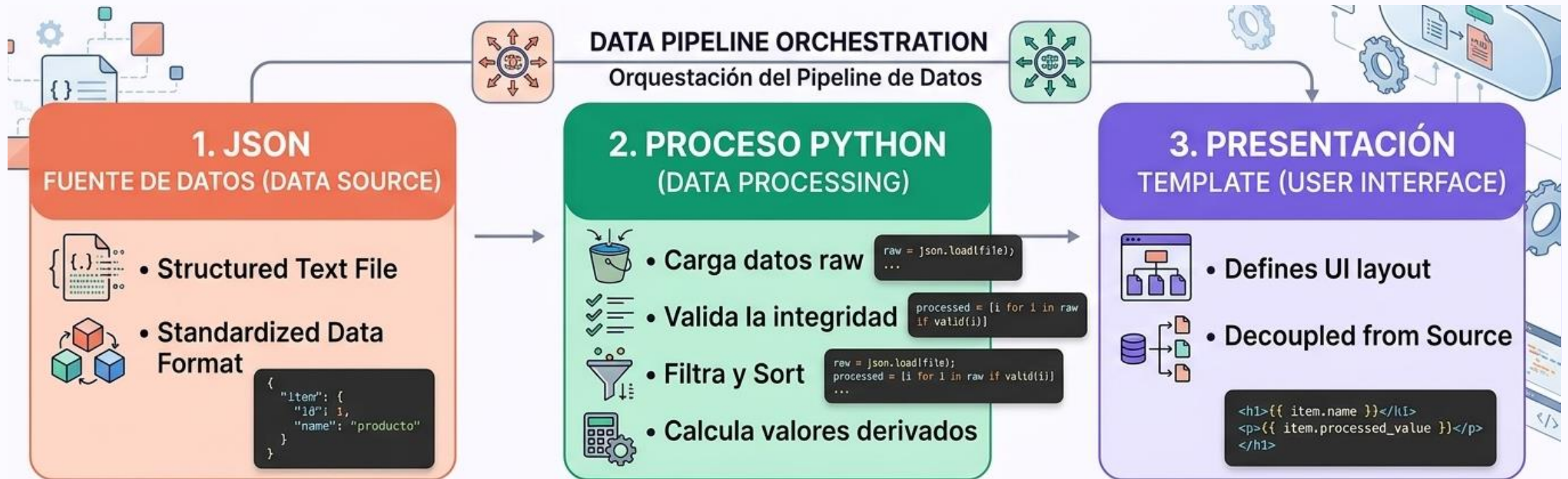
## OBJETIVO DE APRENDIZAJE

**Cargar una colección JSON con Python, enviarla a un Template y utilizar una dependencia externa con una justificación verificable.**

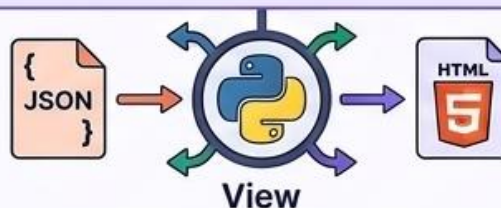
## Al finalizar podrás...

- 01 Construir un JSON válido y coherente.
- 02 Cargarlo mediante una función Python.
- 03 Enviar colección y resumen al Template.
- 04 Instalar, registrar y justificar un paquete externo.

# Cada formato cumple una responsabilidad distinta



**La View coordina el paso desde una fuente JSON hacia estructuras Python y entrega al Template un contexto comprensible.**



# La estructura se mantiene; cambia dónde viven los datos

## LISTA EN `views.py`

- Rápida para una primera prueba.
- Mezcla datos y coordinación.
- Cada cambio modifica código.
- Difícil de reutilizar.

## ARCHIVO JSON

- Separa datos del flujo de la View.
- Puede editarse y reemplazarse.
- Mantiene una estructura explícita.
- Prepara futuras integraciones.

**JSON mejora la separación, pero todavía no reemplaza una base de datos.**

## Paso 1: crea una colección JSON válida

Cree /data/dispositivos.json

```
[
  {
    "id": 1,
    "nombre": "Medidor inteligente",
    "estado": "Activo",
    "consumo_kwh": 18.4
  },
  {
    "id": 2,
    "nombre": "Climatizador",
    "estado": "Revisión",
    "consumo_kwh": 32.7
  }
]
```

### Observa

- Las claves y textos utilizan comillas dobles.
- Cada elemento conserva las mismas claves.
- El último elemento no lleva coma final.

## Paso 2: una función transforma JSON en Python

```
# dispositivos/services.py

import json

from django.conf import settings

def cargar_dispositivos():
    ruta = settings.BASE_DIR / "data" / "dispositivos.json"

    with ruta.open(encoding="utf-8") as archivo:
        datos = json.load(archivo)

    if not isinstance(datos, list):
        raise ValueError("Se esperaba una lista de dispositivos")
    return datos
```

### Observa

- settings.BASE\_DIR construye una ruta estable.
- with cierra el archivo automáticamente.
- json.load devuelve estructuras Python.

## Paso 3: la View carga, calcula y prepara contexto

```
# dispositivos/views.py

def catalogo(request):
    dispositivos = cargar_dispositivos()

    activos = sum(
        1 for item in dispositivos
        if item["estado"] == "Activo"
    )

    contexto = {
        "dispositivos": dispositivos,
        "total": len(dispositivos),
        "total_activos": activos,
    }

    return render(
        request, "dispositivos/catalogo.html", contexto
    )
```

### Observa

- La fuente se carga una vez.
- Python cuenta elementos que cumplen la condición.
- El contexto mantiene nombres claros.

## Paso 4: el Template presenta datos y resumen

```
{% block content %}
  <h1>Dispositivos</h1>
  <p>Total: {{ total }} | Activos: {{ total_activos }}</p>
  {% for dispositivo in dispositivos %}
    <p>{{ dispositivo.nombre }} -
      {{ dispositivo.consumo_kwh }} kWh -
      {{ dispositivo.estado }}</p>
  {% empty %}
    <p>No existen dispositivos disponibles.</p>
  {% endfor %}
{% endblock %}
```

### Observa

- El Template conserva la herencia.
- Las métricas provienen del contexto.
- for representa cualquier cantidad de elementos.

# El error indica si falló el archivo, la sintaxis o la estructura

## SEÑAL

- FileNotFoundError.
- JSONDecodeError.
- Se esperaba una lista.
- Variable vacía en HTML.

## PRIMERA COMPROBACIÓN

- Ruta, carpeta y nombre del archivo.
- Comillas, comas, llaves y corchetes.
- Elemento raíz del JSON.
- Claves del contexto y del registro.

**Corrige una causa a la vez y vuelve a ejecutar la misma prueba.**

# No todo import requiere instalar un paquete

## BIBLIOTECA ESTÁNDAR

- Incluida con Python.
- Ejemplo: json.
- No se instala con pip.
- No agrega una dependencia al proyecto.

## PAQUETE EXTERNO

- Se obtiene fuera de Python.
- Ejemplo: django-bootstrap5.
- Se instala y comprueba.
- Debe registrarse y justificarse.

**Agrega una dependencia solo cuando resuelve una necesidad que el proyecto realmente tiene.**

# Una dependencia se justifica con cuatro evidencias



También debe quedar registrada en `requirements.txt` para que otra persona pueda instalar el mismo entorno.

## Paso 5: instala, registra y utiliza django-bootstrap5

```
python -m pip install django-bootstrap5
python -m pip check
python -m pip freeze > requirements.txt
```

```
# config/settings.py → INSTALLED_APPS
"django_bootstrap5",
```

```
<!-- templates/base.html -->
{% load django_bootstrap5 %}

<head>{% bootstrap_css %}</head>
```

```
<body class="container py-4">
    {% block content %}{% endblock %}
    {% bootstrap_javascript %}
</body>
```

### Observa

- pip instala dentro de .venv.
- INSTALLED\_APPS registra la integración.
- Las etiquetas cargan recursos Bootstrap.

# IA como apoyo: propone el diseño, tú controlas el Template

Diseña el contenido de `catalogo.html` para EcoEnergy.

Usa Bootstrap 5 y hereda de `base.html`.

Variables disponibles:

- `dispositivos`: lista con nombre, estado y `consumo_kwh`
- `total`: cantidad de dispositivos
- `total_activos`: cantidad en estado Activo

Requisitos visuales:

- dos tarjetas de resumen
- tabla responsive
- badge de estado
- estado vacío

No inventes variables, rutas ni lógica Python.  
Devuelve solamente el bloque content.

## Observa

- La IA recibe el contrato real del contexto.
- El estudiante integra etiquetas Django y herencia.
- Cada variable y componente debe comprobarse.

# Laboratorio: integra una colección JSON verificable

## SITUACIÓN

**Migra una colección de tu Proyecto Integrado o de EcoEnergy desde views.py hacia JSON, genera un resumen y utiliza IA para proponer un diseño Bootstrap que puedas comprobar y justificar.**

**1**

**Crea un JSON con al menos cuatro registros coherentes.**

**2**

**Implementa una función de carga fuera de la View.**

**3**

**Calcula al menos un resumen con Python.**

**4**

**Entrega a la IA solamente variables y requisitos reales del Template.**

**Entrega individual: JSON, loader, View, prompt, Template corregido, tres decisiones de diseño, requirements.txt y enlace al commit.**

# La IA ayuda a actualizar un README reproducible

Actualiza el README.md de mi proyecto Django con información comprobada.

Archivo JSON: [RUTA Y ESTRUCTURA]  
Función de carga: [ARCHIVO Y FUNCIÓN]  
Ruta funcional: [URL]  
Dependencia externa: [PAQUETE]  
Justificación y prueba: [NECESIDAD + EVIDENCIA]

Incluye instalación desde requirements.txt y verificación.

No inventes comandos, archivos ni funcionalidades.

Si falta información, pregúntame antes de

## Observa

- Entregar rutas y resultados reales.
- Comprobar cada comando antes de publicarlo.
- Explicar la necesidad del paquete.
- Excluir credenciales y datos personales.

redactar.

# Ticket de salida: explica el recorrido y la decisión

## TICKET DE SALIDA

**¿Cómo llega un dispositivo desde dispositivos.json hasta catalogo.html y por qué django-bootstrap5 sí debe aparecer en requirements.txt?**

**Responde  
brevemente**

**1. Respuesta individual en cinco pasos + justificación de la dependencia + enlace al commit.**

## PRÓXIMA CLASE

**Clase 6: integración MVT, depuración y preparación de la evidencia de Unidad 1.**